ICT912 Programming

Topic 3

Decision Control Structures

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 2:

- Declare, initialize and manipulate variables
- Explain significance of primitive data types
  - Demonstrate operational skills with mathematical operators and compound operators
- Identify and manipulate character data and string data
  - Use built-in functions to manipulate strings
- Explain meanings of coercion and typecasting in the context of data types

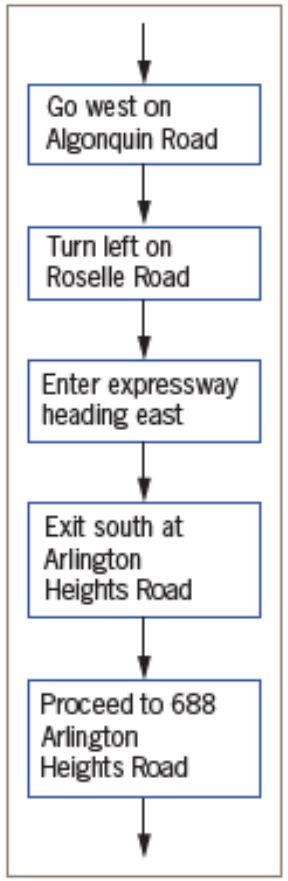
Week 3: Lesson Learning Outcomes

- Describe the purpose of control structures
- Identify and define relational operators
- Explain if-then, if-then-else, nested-if, else-if decision structures
  - Illustrate using flowcharts
- Identify logical operators in conditional statements

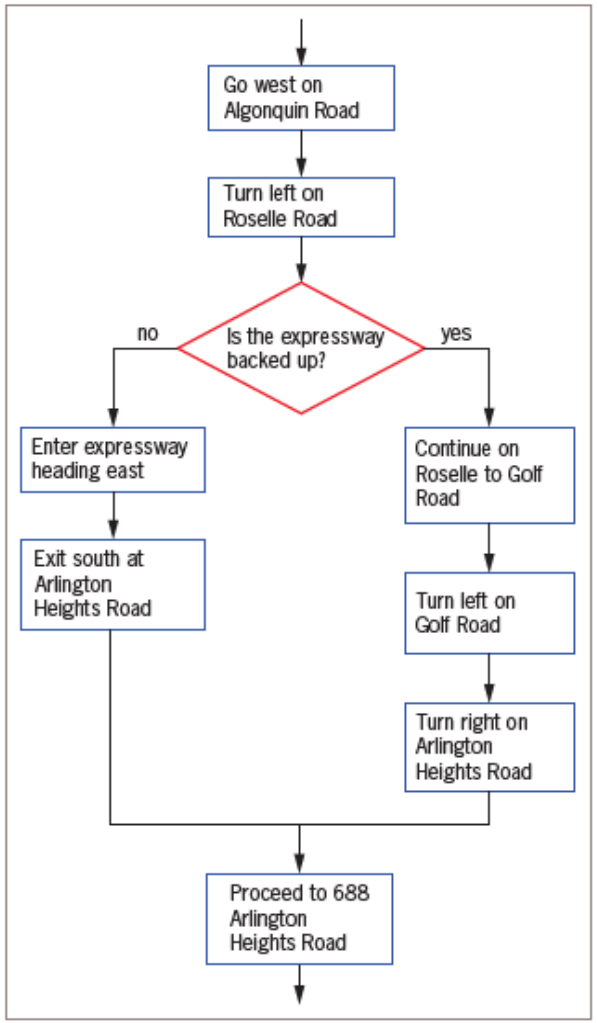
Purpose of Control Structures

- Control Structure:
  - Alters the sequential execution of statements in a computer program
- Decision Control Structure:
  - Alters the sequential flow by branching to specific statements based on a condition or decision

- Decision Logic
  - Help programs seem intelligent because they can make responses that correspond to user input



Sequential flow



Decision branching flow

If-Then Structure

- Logic that directs the program flow to one or more statements in a conditional block when the condition is `True`, or bypass the conditional block if condition is `False`

- Based on a conditional statement that directs the program flow to the correct set of statements when the condition is `True`

- Syntax and Example

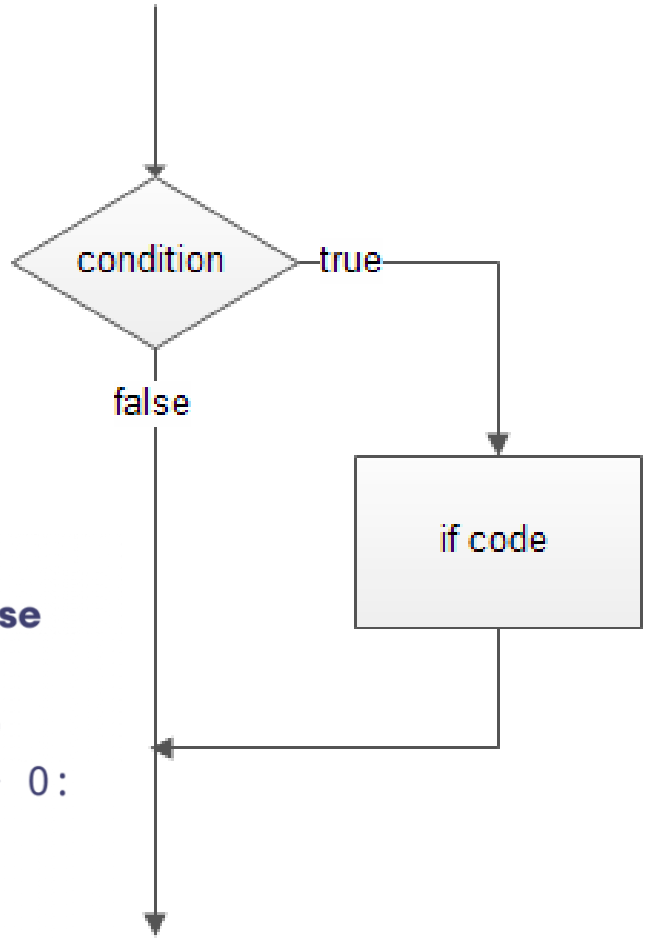
```
if condition:
    statement(s)
```

Condition is True

```
number = 10
if number > 0:
    # code

# code after if
```

Condition is False

```
number = -5
if number > 0:
    # code

# code after if
```



Syntax and Example

Syntax:

```
if condition:
    statement
if condition:
    statement1
    statement2
```

Rules and best practices:

- Place a colon after the condition.
- Indent any statements included in the if-then structure.
- Make sure to keep all statements to be contained within the if-then indented the same amount.

```python
score = 85

if score >= 75:
    print("You have passed your exam!")
    print("Congratulations!")

print("End of program.")
```

```
You have passed your exam!
Congratulations!
End of program.

=== Code Execution Successful ===
```

Indentation in Python

- Python uses indentation to define a block of code, such as the body of `if` statement.

- Example

```python
x = 1
total = 0

# start of the if statement
if x != 0:
    total += x
    print(total)
# end of the if statement

print("This is always executed.")
```

```python
# Error code
x = 1
total = 0

if x != 0:
total += x
print(total)
```

Relational Operators

- The equal operator (==)
  - Checks to see if two operands are equal to each other
  - Different than the assignment operator (=), which is used to initialise variables

```
choice = 1
if choice = 0:
    print("choice is zero")
if choice == 1:
    print("choice is one")
```

Incorrect syntax

Correct syntax

Contd…

- Other relational operators:
  - Specifies comparison between two operands
  - Operands may be literals, variables or one of each

Operator	Description	Example	Result
==	Equal to	5==2	False
!=	Not equal to	5!=2	True
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True

```python
# Declare a variable
favorite_fruit = "apple"

# Check if the favorite_fruit is not equal to "banana"
if favorite_fruit != "banana":
    print("Your favorite fruit is not a banana.")
```

Output

```
Your favorite fruit is not a banana.

=== Code Execution Successful ===
```

Boolean Expression and Data Types

- Expressions formed from relational operators and operands that evaluate to `True` or `False`

- Boolean expression is a part of a conditional statement, whereas Boolean data type is used when making declaration or assignment statements

- In memory, Boolean variables hold 0 representing `False` , or 1 representing `True`

- Example:

```python
# Declare a boolean variable
is_active = True

# Use the boolean variable in an if statement
if is_active:
    print("The user account is active.")
```

Output

```
The user account is active.

=== Code Execution Successful ===
```

If-Then-Else Structure

- A decision coded as two possible paths
- The `else` keyword designates the alternative execution path
- Syntax and Example

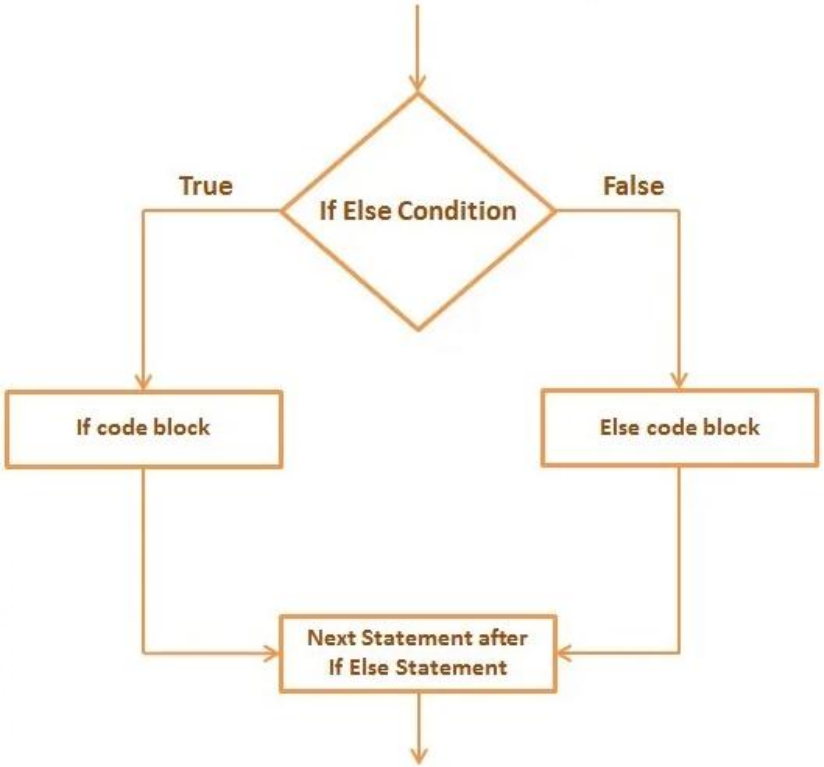
```
if condition:
    statement(s)
else:
    statement(s)
```

Condition is True

```
number = 10
if number > 0:
    # code

else:
    # code

# code after if
```

Condition is False

```
number = -5
if number > 0:
    # code

else:
    # code

# code after if
```



Syntax and Example

Syntax:

```
if condition:
    statement1
    statement2
else:
    statement3
    statement4
```

Rules and best practices:

- Place the **else** keyword on a separate line and at the same indent level as the **if** statement.
- Use a colon immediately after the **else** keyword with no condition statement.

```
number = int(input('Enter a number: '))

if number > 0:
    print('Positive number')
else:
    print('Not a positive number')

print('This statement always executes')
```

```
Enter a number: -5
Not a positive number
This statement always executes

=== Code Execution Successful ===
```

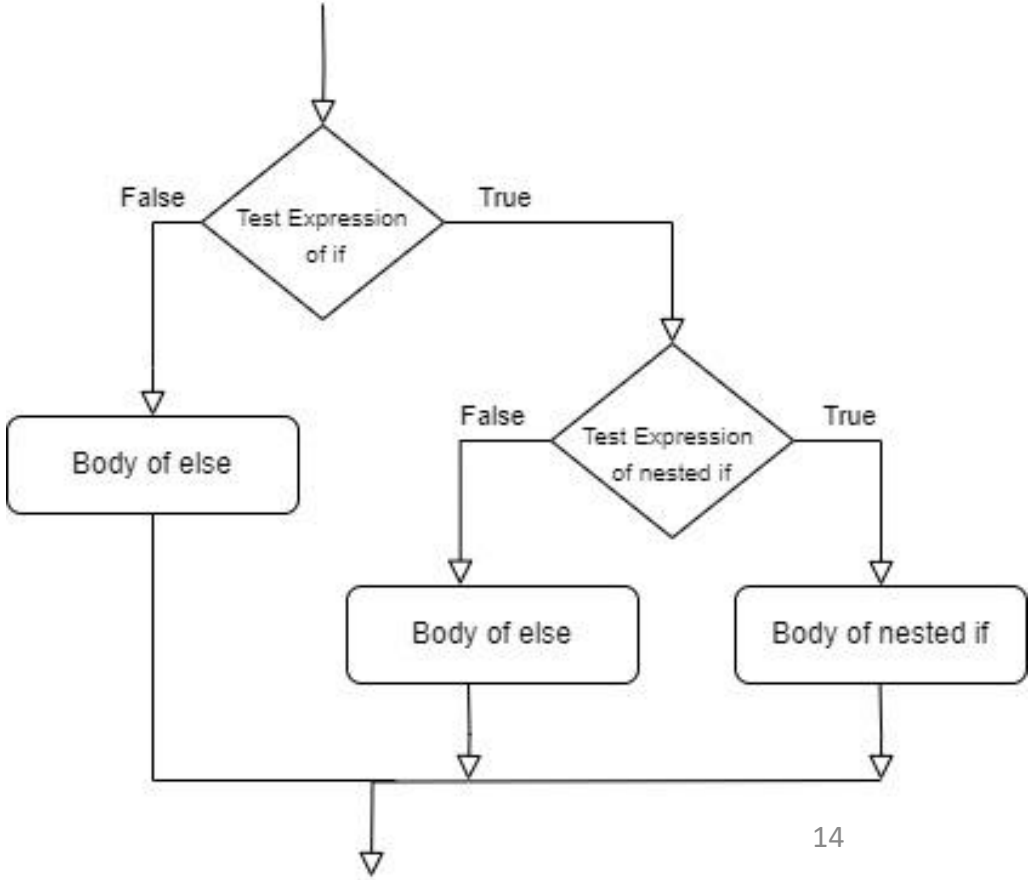
```
Enter a number: 10
Positive number
This statement always executes

=== Code Execution Successful ===
```

Nested-If Structure

- Decisions within decisions
- To implement, use a second indent level for the inner decision structure

- Syntax and Example:

```
if condition1:    # outer if
    if condition2: # inner if
        statement(s)
    else:                # inner else
        statement(s)
else:                    #outer else
    statement(s)
```



Examples

Outer if Condition is True

```
number = 5
if number >= 0:
  if number == 0:
    #code
  else:
    #code
else:
  #code
```

Outer if Condition is False

```
number = -5
if number >= 0:
  if number == 0:
    #code
  else:
    #code
else:
  #code
```

```
number = 5

# outer if statement
if number >= 0:
    # inner if statement
    if number == 0:
      print('Number is 0')

    # inner else statement
    else:
        print('Number is positive')

# outer else statement
else:
    print('Number is negative')
```

Output

```
Number is positive

=== Code Execution Successful ===
```

Else-If Structure

- Structure used to handle algorithms that require multiple conditions
- Combines the two keywords `if` and `else` into a new keyword to represent this concept

- Syntax and Example:

1st Condition is True

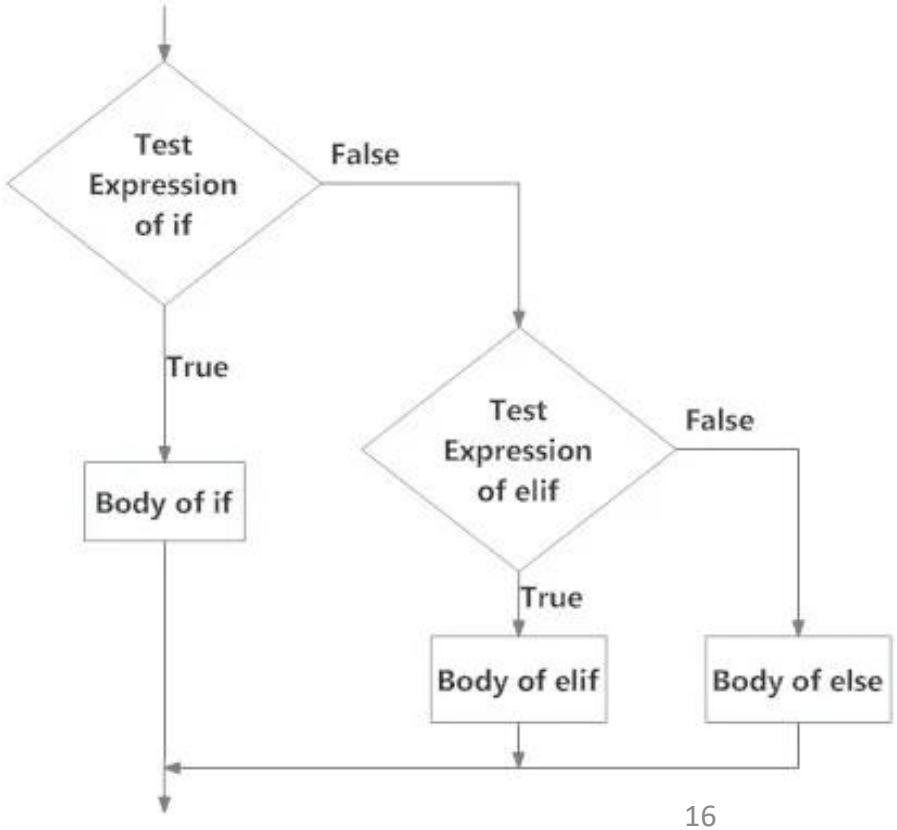
```
let number = 5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

2nd Condition is True

```
let number = -5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

All Conditions are False

```
let number = 0
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```



Examples

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
elif condition3:
    statement3
    statement4
else:
    statement5
```

Rules and best practices:

- Begin with an **if** statement.
- Use the keyword **elif** for subsequent conditional statements.
- Use the keyword **else** for the final condition.

```python
number = -5

if number > 0:
    print('Positive number')

elif number < 0:
    print('Negative number')

else:
    print('Zero')

print('This statement is always executed')
```

Output

```
Negative number
This statement is always executed

=== Code Execution Successful ===
```

Conditional Operators

- Conditional logical operator:
  - The operators `and` and `or`, which combine the outcomes of two or more Boolean expressions

- Truth Table:
  - A table summarizing the possible outcomes for logical operations

Operator	Description
`and`	Returns `true` if both statements are `true`
`or`	Returns `true` if one or all the statements are `true`
`not`	Reverse the result, returns `false` if statement is `true`

and Operator

- Requires both the conditions to be true for the expression to evaluate to `True`

```python
age = 25
has_license = True

if age >= 18 and has_license:
    print("You are eligible to drive.")
else:
    print("You are not eligible to drive.")
```

Output

```
You are eligible to drive.

=== Code Execution Successful ===
```

Where:
a = result of first condition (age>=18)
B = result of second condition (has_license)

Truth Table

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

or Operator

- Requires only one of the condition is true
- When writing a Boolean expression with multiple logical operators, use parentheses to ensure the logic is carried out in the correct sequence

```python
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("It's a day off!")
else:
    print("It's a workday.")
```

Output

```
It's a day off!

=== Code Execution Successful ===
```

Where:
a = result of first condition (is_weekend)
b = result of second condition (is_holiday)

Truth Table

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

not Operator

- Negation operator
- Flips any Boolean value to other

Where:
a = result of the condition (is_raining)

Truth Table

a	not(a)
True	False
False	True

```python
is_raining = False

if not is_raining:
    print("Let's go for a walk!")
else:
    print("Stay indoors.")
```

Output

```
Let's go for a walk!

=== Code Execution Successful ===
```

Next Week – What's Coming Up?

- Understand the need for the repetition control structure
- Explore various types of repetition control structures
  - for loop, nested loop, pre-test loops
- Advantages and disadvantages of the various structures

CIHE

CIHE Australia